



Technical Assessment: Resource Allocation Engine

Overview

Build a **Resource Allocation Engine** that optimally assigns mobile resources to incoming requests. This is a core logistics/supply chain optimization problem.

You have flexibility in how you approach and scope the problem - we're interested in seeing your problem-solving approach, code quality, and ability to build a complete working solution.

Problem Statement

You have a set of **resources** at various locations with different capabilities and schedules. You receive **requests** that need to be fulfilled, each specifying a location, time, and requirements. Your task is to build a system that assigns resources to requests in an optimal manner.

Example Domains (Choose One or Create Your Own)

- **Delivery Fleet:** Trucks assigned to delivery orders
- **Field Service:** Technicians assigned to repair requests
- **Healthcare:** Medical staff assigned to patient visits
- **Equipment Rental:** Equipment assigned to job sites

Feel free to pick a domain that interests you or create your own scenario.

Requirements

Core Functionality

1. Data Model

Design appropriate data structures for your chosen domain. At minimum, you'll need to represent:

- Resources (with location, availability, capabilities)
- Requests (with location, requirements, timing)
- Assignments (resource-request pairings)

2. Allocation Algorithm(s)

Implement **at least two** different allocation strategies to compare their effectiveness. Some options:

Algorithm	Description
Greedy	Process requests sequentially, assigning the best available resource for each
Hungarian	Use the Hungarian/Kuhn-Munkres algorithm for optimal batch assignments
Custom Heuristic	Your own approach based on domain insights
ML-Based	Learn allocation patterns from data

Your implementation should:

- Handle relevant constraints for your domain
- Provide some form of decision explanation
- Report meaningful metrics

3. Constraints

Implement constraints appropriate to your domain. Consider both:

- **Hard constraints:** Must be satisfied (e.g., availability, capability match)
- **Soft constraints:** Preferences that affect scoring (e.g., minimize distance)

4. Testing

Include tests that demonstrate your algorithms work correctly and show meaningful comparisons between approaches.

Visual Interface (Required)

Build a **web-based UI** that visualizes your solution. This should include:

- **Map/Spatial View:** Show resources, requests, and assignments visually
- **Algorithm Comparison:** Display results from different algorithms side-by-side
- **Metrics Display:** Show relevant performance metrics

Interactive features (adding resources, adjusting parameters, etc.) are a plus but not required.

Documentation

- **README:** Setup instructions and overview of your approach
- **Brief Analysis:** What did you learn from comparing algorithms?



Tech Stack (Required)

Layer	Technology
Frontend	React
Backend	FastAPI or Django
Other Python libs	Your choice (NumPy, SciPy, etc.)

Important Constraints

- **No paid services:** Use only free/open-source tools and libraries. For maps, use free options like Leaflet with OpenStreetMap, or a simple canvas/SVG visualization.
- **Run locally:** The solution must work entirely on a local machine without requiring API keys or external paid services.

Deliverables

1. Source code (Git repository or zip file)
2. Working web application with visualization
3. README with setup instructions
4. Test suite
5. Brief write-up comparing your algorithms

Submission Guidelines

- **Exclude `node_modules`** folder from your submission
- Include a `requirements.txt` or `pyproject.toml` for Python dependencies
- Include `package.json` for frontend dependencies (reviewers will run `npm install`)
- Ensure the application can be started with clear instructions in the README

Example: Algorithm Comparison Insight

One interesting comparison: A greedy algorithm processes requests one-by-one and makes locally optimal choices. A batch algorithm like Hungarian considers all assignments simultaneously and can find globally better solutions. When might each approach win? When does the difference matter most?

Flexibility

We've intentionally left many details open-ended. We want to see how you think through the problem and make reasonable decisions.
